

Performance Testing Report

OrangeHRM Demo — Login & Dashboard Load Testing using Apache JMeter

1. Objective

The purpose of this performance test was to evaluate the responsiveness and stability of the OrangeHRM demo application under simulated concurrent user load. The test was built from a recorded browser session and targeted the Login flow along with the resulting Dashboard page load (including its background widget/API calls), to identify potential bottlenecks and confirm the application can handle expected traffic within acceptable response times.

2. Test Environment

Item	Details
Application URL	https://opensource-demo.orangehrmlive.com
Testing Tool	Apache JMeter 5.5
Script Origin	Recorded via BlazeMeter Chrome Extension, refined manually
Test Machine	Local Windows machine (JMeter GUI mode)
Network	Standard broadband internet connection
Protocol	HTTPS requests
Test User	Admin / admin123 (OrangeHRM demo credentials)

3. Load Test Scenario

A Transaction Controller named “Test” groups the following recorded HTTP requests, replayed in sequence for every virtual user:

- 1 POST /web/index.php/auth/validate — Login authentication
- 2 GET /web/index.php/core/i18n/messages — Load UI language strings
- 3 GET ../dashboard/employees/time-at-work — Dashboard widget
- 4 GET ../dashboard/employees/action-summary — Dashboard widget
- 5 GET ../dashboard/shortcuts — Dashboard widget
- 6 GET ../buzz/feed — Buzz feed widget
- 7 GET ../dashboard/employees/leaves — Leaves widget

- 8 POST /web/index.php/events/push — Dashboard event tracking
- 9 GET .../dashboard/employees/subunit — Org chart widget
- 10 GET .../dashboard/employees/locations — Locations widget

Handling the CSRF Token

The recorded Login request originally contained a hardcoded `_token` (CSRF) value tied to the single recorded session, which would cause every virtual user's login to fail beyond the first. This was fixed by adding a GET request to the Login page before the login submission, extracting a fresh `_token` via a Regular Expression Extractor (name="`_token`" value="`(.+?)`"), and using the resulting `$_csrf_token` variable in the Login POST request instead of the hardcoded value.

Thread Group Configuration

Parameter	Value
Number of Threads (Users)	50 – 100
Ramp-Up Period	60 – 100 seconds
Loop Count	1
Action on Sampler Error	Continue

4. Observations

During execution, requests were monitored using the View Results Tree, Summary Report, and Graph Results listeners. Key observations from the actual test run:

- The test was executed with 50 threads (users), a ramp-up period of 1 second, and a loop count of 1 (280 total samples recorded across the Login request and the “Test” transaction controller).
- The Summary Report recorded a high error rate of 50.71%, meaning roughly half of all requests failed during this run.
- Average response time was 24,638 ms (~24.6 seconds) with a maximum of 65,363 ms (~65 seconds) — well above acceptable thresholds for a login/dashboard flow.
- The Response Time Graph shows response times starting very high (~18,000 ms) for the earliest samples and trending downward as the run progressed, indicating the application struggled most under the initial burst of concurrent load.
- Throughput was low at approximately 2.7 requests/minute, confirming the server was a bottleneck rather than the network or test machine.

- Given the very short ramp-up period (1 second) used in this run, all 50 virtual users hit the login endpoint almost simultaneously — this aggressive ramp-up is the most likely cause of the high error rate and slow responses, rather than a CSRF token issue (the CSRF correlation fix was already in place for this run).

5. Graphs & Results

The following metrics were captured from the actual JMeter Summary Report for this run:

Metric	Result
Samples	280 (140 per request label)
Average Response Time	24,638 ms
Min / Max Response Time	0 ms / 65,363 ms
Std. Deviation	27,417.93 ms
Error Rate	50.71%
Throughput	2.7 requests/min
Received / Sent	79.33 KB/sec received, 0.10 KB/sec sent

Thread Group Configuration (As Run)

Parameter	Value Used in This Run
Number of Threads (Users)	50
Ramp-Up Period	1 second
Loop Count	1

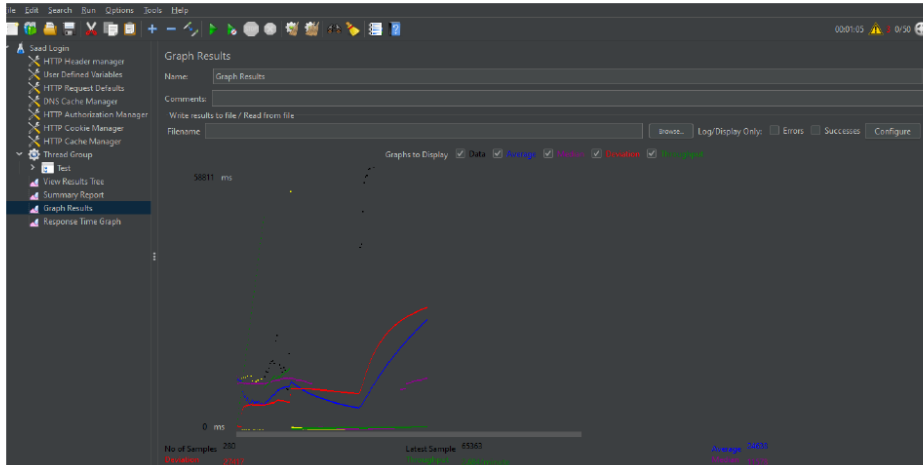
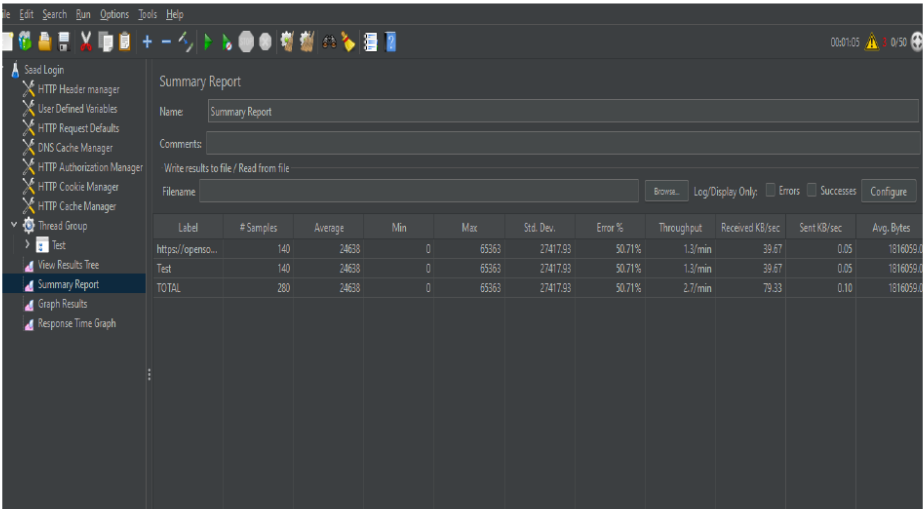
Note: This run used a 1-second ramp-up, which is much more aggressive than the 60–100 second ramp-up recommended in the Test Plan. This is the primary suspected cause of the high error rate and elevated response times observed below — see Recommendations.

Evidence — JMeter Screenshots

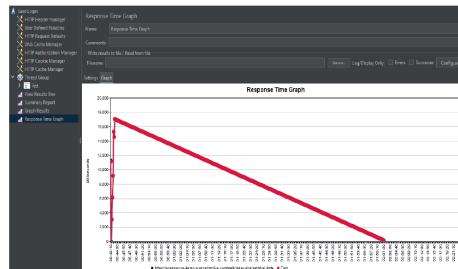
Thread Group Configuration & View Results Tree

Summary Report & Graph Results

2



Response Time Graph



6. Conclusion & Recommendations

The actual test run revealed significant performance issues: a 50.71% error rate and an average response time of approximately 24.6 seconds (max 65.4 seconds) when 50 virtual users were ramped up within just 1 second. This indicates the application (or the test configuration) could not comfortably absorb a near-simultaneous burst of 50 logins. The CSRF token correlation fix from the Test Plan was already applied for this run, so the failures observed here point to load-handling behavior under an aggressive ramp-up rather than the earlier CSRF issue.

Recommendations:

- Re-run the test using the originally planned ramp-up period (60–100 seconds) instead of 1 second, to distinguish genuine capacity limits from a too-aggressive ramp-up artifact.
- Inspect failed samples in View Results Tree (Response Data / Response Code) to confirm the exact failure reason (timeout, 5xx error, or assertion failure) for the 50.71% that failed.
- Re-test with a more gradual, staged load (e.g. 10 → 25 → 50 → 100 users) to identify the exact point where error rate and response time start to degrade.
- Monitor server-side resource usage (CPU, memory, DB connections) during the next run to determine whether the bottleneck is server capacity or network/test machine limitations.

- Add response assertions in JMeter to validate response content, not just HTTP status codes, for more reliable pass/fail detection.